`shutil` and `zipfile`

Status: not covered by Software Lab-1 practicals **Where it is used:** `backend/store.py` (`backup_all` — `shutil.copyfile`), `scripts/stage_datasets.py` (`stage_pima` — `shutil.copyfile`; imports `zipfile` but never calls it), `tests/test_store.py` and `tests/test_app.py` (`shutil.rmtree`) **Closest lab concept:** Practical 4, functions and files — `os` moves one file at a time; `shutil` copies whole files and directory trees; `zipfile` packs and unpacks archives

---

1. What they are

Two stdlib modules, both covering ground `open()` does not.

`shutil` stands for shell utilities. Lab 4 taught you to copy a file by opening the source, reading it and writing the contents into a new `open()` handle — three lines of buffering logic that you have to get right, and that get slow on large files because Python moves the data in small chunks. `shutil` does it in one call, efficiently, for one file or a whole directory tree.

`zipfile` reads and writes ZIP archives — the `.zip` format Windows sends you when you right-click a folder. Python ships with it, so extracting a downloaded archive needs no `pip install` at all.

Both are stdlib, exactly like `os`, `math` and `csv`. The difference from `os` is scope: `os` knows about *paths* (does this exist, list this folder, join these parts, walk this tree). `shutil` knows about *file contents and trees*. `zipfile` knows about *archives*.

---

2. Why the project needed them

Two separate jobs, both of which Lab 4 code does badly.

Bulk copies. `store.backup_all()` snapshots the five registry CSVs into a timestamped folder. Copying five files by hand means five open / read / write loops, and the same duplication appears in `scripts/stage_datasets.py` when it promotes a downloaded file into place. `shutil.copyfile` collapses each of those to one line.

Archives. Research datasets do not arrive as tidy CSVs. The UCI Machine Learning Repository — the project's canonical source — hands out `.zip` files, and what is inside is frequently a `.data` file with **no header row at all**. `scripts/stage_datasets.py` exists precisely to deal with that, and `zipfile` is the tool that gets you inside the archive (section 6).

---

3. The pieces actually used

3.1 `shutil.copyfile(src, dst)` in `store.backup_all()`

The whole copy operation, `store.py:278-287`:

```
    for record_type in RECORDS:
        filename = RECORDS[record_type][0]
        source = os.path.join(DATA_DIR, filename)

        if not os.path.exists(source):
            continue

        target = os.path.join(destination, filename)
        shutil.copyfile(source, target)
        written.append(target)
```

Note that `shutil` takes *paths*, not file handles — you pass `source` and `target` strings built with `os.path.join`, and never open anything yourself. The `os.path.exists(source)` check before it matters: on

a fresh checkout not every registry file exists yet, and `shutil.copyfile` raises `FileNotFoundError` rather than skipping, so the guard is what lets a partial `data/` folder back up cleanly.

The destination folder is created with `os.makedirs` (`store.py:273-274`), and the timestamp comes from Practical 4's `datetime` work (`store.py:270`):

```
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    destination = os.path.join(BACKUP_DIR, stamp)

    if not os.path.exists(destination):
        os.makedirs(destination)
```

`%f` is microseconds. That is deliberate: `test_backup_twice_does_not_collide` (`tests/test_store.py:256`) calls `backup_all()` twice in a row and asserts two separate folders, which a second-resolution timestamp could not guarantee.

3.2 `shutil.copyfile` again, in `stage_datasets.py`

`stage_pima`, `scripts/stage_datasets.py:62-68`:

```
def stage_pima():
    """Plotly's copy already carries the standard Pima header."""
    src = os.path.join(RAW, "pima_diabetes_local.csv")
    dst = os.path.join(STAGED, "pima_diabetes.csv")
    if os.path.exists(src):
        shutil.copyfile(src, dst)
        print(f"  staged pima_diabetes.csv")
```

Same call, different reason. This dataset already has a correct header, so staging is a pure rename-by-copy — no rewriting needed. The other three datasets go through `write_with_header` instead (`scripts/stage_datasets.py:52-59`), because they arrive headerless:

```
def write_with_header(name, columns, lines):
    """Write lines to a CSV, prefixed with a header row."""
    path = os.path.join(STAGED, name)
    with open(path, "w", newline="") as f:
        f.write(",".join(columns) + "\n")
        for line in lines:
            f.write(line + "\n")
```

3.3 `shutil.rmtree(path)` in the tests

Used to clean up after tests that write real folders, `tests/test_store.py:236`:

```
    shutil.rmtree(store.BACKUP_DIR, ignore_errors=True)
```

`rmtree` deletes a directory and everything under it. `os` cannot do this — `os.remove` refuses a non-empty directory, and `os.removedirs` walks *upward* removing empty parents, which is the wrong direction when you want one known subtree gone. `ignore_errors=True` makes a missing folder a no-op instead of an exception, so the same cleanup line works both before and after a test run.

---

4. `copyfile` vs `copy` vs `move`

The distinction is narrow but real, and picking the wrong one is a common interview slip.

Call	Copies contents	Copies permission bits	Removes the source	Whole directory tree
`shutil.copyfile(src, dst)`	yes	no	no	no
`shutil.copy(src, dst)`	yes	yes	no	no

`shutil.copytree(src, dst)`	yes	yes	no	yes
`shutil.move(src, dst)`	yes, if needed	yes	**yes**	yes

- **`copyfile`** — content only. The destination gets default permissions rather than the source's mode bits.
- **`copy`** — `copyfile` plus `copymode`, so an executable script stays executable. This project has no executables, which is why `copyfile` is the right choice here.
- **`move`** — renames when both paths are on the same filesystem, and falls back to copy-then-delete across filesystems. **Watch the destination:** if `dst` is an existing *directory*, the source lands *inside* it rather than replacing it. That silent "it ended up one level deeper than I expected" behaviour is why `backup_all` uses `copyfile` and builds its own destination path — a backup must never mutate the live file, and `move` would delete the original.
- **`copytree`** — the one to reach for when the thing you are copying is a folder, not a file. Not used in this project, but it is what `rmtree`'s counterpart on the copy side would be.

Rule of thumb: **use `copy` unless you specifically want to skip permissions, and use `copyfile` only when you are deliberately not carrying the mode.** Never use `move` for a backup.

---

5. Gotcha: the `os.walk` backup bug, and the fix

This is a real bug this project hit, and the surviving evidence is unusually good — the reason is written into the function's own docstring (`store.py:256-265`):

```
def backup_all():
    """Copy the five registry CSVs into a timestamped backup folder.

    Scope is deliberately limited to the registries. data/datasets/ holds
    downloaded research data that never changes at runtime, and it contains
    filenames that collide with the registry files once flattened into a
    single folder, so including it would silently overwrite one with the
    other.

    Returns the list of destination paths written.
    """
```

What went wrong. The first version discovered files with `os.walk(DATA_DIR)` — correct, it just visited every directory under `data/`, which includes `data/datasets/` and `data/datasets/raw/`. For each file it computed the destination with `os.path.basename(path)`. That throws away the folder information, so every file from every subdirectory lands in the *one* backup folder under its bare name.

Any duplicate basename in that walk now collides. Both of these files exist in this repo today:

- `data/datasets/parkinsons.csv` (staged, cleaned header names)
- `data/datasets/raw/parkinsons.csv` (as downloaded)
- `data/datasets/mammographic_masses.csv` and `data/datasets/raw/mammographic_masses.csv` collide the same way

Both copies are written to the same destination path. The second `shutil.copyfile` overwrites the first. No exception is raised, no file is missing, and the backup folder has the right number of files in it — so nothing looks wrong. The backup simply contains the *wrong* version of one file, and if the two had differed in content the loss would be invisible until the day someone needed to restore.

This is the worst class of bug: the failure is silent and the symptom appears far from the cause, at restore time rather than backup time.

The fix. Stop discovering files and name exactly the five you meant (`store.py:278-279`):

```
    for record_type in RECORDS:
        filename = RECORDS[record_type][0]
```

`RECORDS` is the fixed registry of filenames from `store.py:44-50`, so the loop can only ever emit those five names. Nothing is flattened, nothing can collide, and the scope of a backup is stated in the data rather than inferred from the filesystem.

The regression test. `tests/test_store.py:229-253` pins the behaviour so it cannot come back:

```
def test_backup_copies_only_the_registries():
    """Backups must not sweep in the research datasets.

    data/datasets/ holds files whose names collide with registry files, so
    flattening everything into one folder silently overwrites one with
    the other.
    """
    shutil.rmtree(store.BACKUP_DIR, ignore_errors=True)

    written = store.backup_all()

    names = sorted(os.path.basename(p) for p in written)

    assert len(written) == 5, names
    assert names == ["diseases.csv", "medicines.csv", "patients.csv",
                     "stores.csv", "visits.csv"], names
```

The test asserts the exact filename set, not just the count — a count of 5 would still pass if the wrong five files were copied.

The transferable lesson: **`os.walk` plus `os.path.basename` is a lossy round trip.** If you flatten a tree by discarding directory components, you are asserting that no two files in that tree share a basename. In this repository that assertion was false. When it matters, either check for collisions explicitly or, as here, do not flatten at all.

---

6. `zipfile`: getting inside an archive

6.1 What the module gives you

`zipfile.ZipFile(path)` opens an archive for reading (pass `"w"` to create one). The three members this kind of work needs:

- `namelist()` — the member paths stored in the archive, as a list of strings. Always call this first: it is how you find out what is inside without writing anything to disk.
- `read(name)` — the contents of **one** member, returned as `bytes`.
- `extractall(path)` — unpacks every member into a folder.

Reading a single member, without unpacking the archive. This block is **stdlib API written out for illustration, not code that exists in this repository** — the project never gets this far (section 6.2):

```
import zipfile

with zipfile.ZipFile("parkinsons.zip") as z:
    names = z.namelist()
    raw = z.read("parkinsons.data")
text = raw.decode("utf-8").splitlines()
```

That is the pattern that matters for this project: decompress one member in memory, turn it into lines, and **never touch the filesystem**. Compare with the loop `stage_datasets.py` already uses for a plain file (`scripts/stage_datasets.py:77-79`) — the shape is identical, only the source changes:

```
    with open(src) as f:
        lines = [ln.strip() for ln in f if ln.strip()]
```

Swap `open(src)` for `ZipFile(...).read(...)` and the rest of the staging pipeline is unchanged. Nothing is extracted, nothing is deleted afterwards, and no temporary folder needs cleaning up.

`extractall()` is the opposite trade: convenient, but it writes every member to disk at once, including the documentation files and folders you do not want. Use `read()` when you want one file; use `extractall()` when you genuinely want the whole tree. Always prefer the `with` block — it closes the archive deterministically, which is the Lab 4 `with` statement habit applied to a resource that is not a file object.

Two habits worth forming: `namelist()` before acting, so you never assume a member name; and checking whether a member is a directory before extracting, because `extractall` will happily create folders you then have to reason about.

6.2 Where `zipfile` actually stands in this project

Be precise about this, because the code is easy to over-read. `scripts/stage_datasets.py:10` has:

```
import zipfile
```

and that is the **only** occurrence of `zipfile` anywhere in the codebase. It is never called. The archive-handling was planned and the plan was later dropped in favour of plain CSV downloads — `scripts/fetch_datasets.py` fetches every dataset as a flat `.csv` from GitHub raw URLs, so nothing is ever compressed by the time `stage_datasets.py` runs.

The intent is still recorded in the file. `scripts/stage_datasets.py:49`:

```
PARKINSONS_URL = "https://archive.ics.uci.edu/static/public/174/parkinsons.zip"
```

That is a real UCI `.zip` endpoint, and the constant is defined but never used by any function — `stage_parkinsons()` instead expects `raw/parkinsons.csv` to already be on disk.

So the honest summary: **`zipfile` is imported as a record of an intended design, not as working code.** An unused import is a plan, not a feature. If you are asked "where does this project use zipfile?", the correct answer is that it does not yet, and that the leftover import and URL are the fossil of that plan.

6.3 Why UCI archives force the whole staging design

Even setting the unused import aside, the *reason* the design looks like this is worth understanding, and the docstrings say so directly. `scripts/stage_datasets.py:71-72`:

```
def stage_heart():
    """Cleveland heart data: no header, 14 columns, '?' means missing."""
```

and `scripts/stage_datasets.py:82-83`:

```
def stage_masses():
    """Mammographic masses: no header, '?' marks missing values."""
```

UCI ships a `.zip` containing a `.data` file: whitespace-separated, no header line, and `?` standing in for missing values. That is not a CSV by any reasonable definition, and pandas — Practical 6 — will read the first data row as the column names and produce a dataframe with garbage headers. So `stage_datasets.py` writes the correct header itself from the official column lists (`HEART_COLUMNS`, `MASS_COLUMNS`, `scripts/stage_datasets.py:15-22`) and prefixes it to the raw lines.

Parkinsons is the exception that proves the rule (`scripts/stage_datasets.py:93-94`):

```
def stage_parkinsons():
    """Parkinsons already ships a header row; just tidy the column names."""
```

Its header exists, but the names are things like `MDVP:Fo(Hz)` and `PPE`, which are useless as DataFrame

column names. So the same function renames them through a dict, `PARKINSONS_RENAME` (`scripts/stage_datasets.py:25-47`), and re-emits the header:

```
    original_header = lines[0].split(",")
    columns = [PARKINSONS_RENAME.get(c, c) for c in original_header]
    write_with_header("parkinsons.csv", columns, lines[1:])
```

`.get(c, c)` falls back to the original name for anything the dict does not mention, so adding a column upstream cannot crash the script.

Note the contrast with `docs/concepts/csv-module.md`: this code *does* use `split(",")` deliberately, on a single known-clean header line it controls itself. That is the difference between parsing a known file and parsing user-supplied data — the discipline `csv` exists to enforce.

---

7. `os` versus `shutil` versus `zipfile`

Task	Module	Example in this project
Does a path exist?	`os`	`os.path.exists(source)`, `store.py:282`
Build a path	`os`	`os.path.join(DATA_DIR, filename)`, `store.py:280`
Create a directory	`os`	`os.makedirs(destination)`, `store.py:274`
List a directory	`os`	`os.listdir(store.BACKUP_DIR)`, `tests/test_store.py:263`
Walk a tree	`os`	`os.walk` — the bug in section 5
Copy one file	`shutil`	`shutil.copyfile(source, target)`, `store.py:286`
Copy a tree	`shutil`	`shutil.copytree`
Delete a tree	`shutil`	`shutil.rmtree(store.BACKUP_DIR, ignore_errors=True)`, `tests/test_store.py:236`
Read/parse a CSV	`csv`	`csv.DictReader(f)`, `store.py:103`
Read inside a `.zip`	`zipfile`	imported, not yet called, `scripts/stage_datasets.py:10`

The line to remember: **`os` is about paths, `shutil` is about file contents and trees, `zipfile` is about archives.** When you find yourself writing `open()`, a read loop and a write loop to move data around, stop and look for `shutil` first.

---

8. Questions this should answer

- What is the difference between `shutil.copyfile`, `shutil.copy` and `shutil.move`, and why must a backup never use `move`?
- Why does `backup_all` check `os.path.exists(source)` before calling `shutil.copyfile`?
- What did the original `os.walk` version of `backup_all` do wrong, and how does `os.path.basename` cause the failure?
- What does `read(name)` give you that `extractall()` does not, and when should you prefer each?
- Why is `zipfile` imported in `scripts/stage_datasets.py` but never actually called — and what does that tell you about the code?
- Why do UCI `.data` files force `stage_datasets.py` to write its own header row before pandas can read them?